

Laboratório Ataque x Defesa (Força Bruta)

Tópicos Avançados II — IFBA | Relatório técnico sucinto
Repositório: github.com/CaioGabriel777/TRABALHO-INF032

1. Resumo

Este trabalho implementa um laboratório educacional completo para estudo de ataques de força bruta contra autenticação e das defesas correspondentes. O ambiente é composto por uma API de login em Spring Boot (defesa-api), um script/servidor de ataque em Python (ataque-cli / ataque-api) e dois painéis web em Next.js (defesa-web e ataque-web), orquestrados via Docker Compose junto de um banco PostgreSQL. O sistema permite alternar a defesa (rate limiting) em tempo real e comparar, de forma objetiva, o comportamento do alvo sob ataque com e sem proteção.

2. Introdução

Ataques de força bruta consistem em testar sistematicamente diversas combinações de credenciais (geralmente a partir de uma wordlist) contra um mecanismo de autenticação até encontrar uma senha válida. Apesar de conceitualmente simples, seguem sendo uma das causas mais comuns de comprometimento de contas quando o sistema alvo não implementa nenhuma forma de limitação de tentativas.

O objetivo do laboratório é tornar esse comportamento observável e mensurável: de um lado, uma aplicação vulnerável por padrão (mas com hashing de senha correto); de outro, uma ferramenta de ataque restrita por segurança a atacar apenas esse alvo controlado, dentro da rede interna do docker-compose. Os dois lados expõem dashboards que tornam visível, em tempo quase real, o que aconteceria de forma silenciosa em um ataque real.

3. Problema

Um endpoint de login que aceita tentativas ilimitadas, sem qualquer atraso ou bloqueio, permite que um atacante automatizado teste uma wordlist inteira em segundos, sem ser notado nem impedido. Isso é agravado por dois fatores comuns em sistemas reais:

- Ausência de limite por IP e por usuário, permitindo tentativas ilimitadas a qualquer taxa;
- Falta de visibilidade: sem logging estruturado das tentativas, o ataque só é percebido depois do estrago (conta comprometida);

O desafio, portanto, não é apenas "bloquear tentativas", mas fazer isso sem comprometer usuários legítimos, mantendo o custo de implementação simples e o comportamento auditável — e, para fins didáticos, permitindo comparar o "antes" e o "depois" da defesa no mesmo sistema.

4. Solução

A defesa foi implementada em defesa-api como um rate limiter em memória, com janela deslizante, aplicado tanto por IP quanto por nome de usuário (RateLimiterService). Cada tentativa de login malsucedida é registrada com timestamp; entradas mais antigas que a janela configurada são descartadas a cada verificação, e o acesso é bloqueado assim que o número de falhas recentes atinge o limite configurado (padrão: 5 tentativas em uma janela de 60 segundos).

Outros elementos da solução:

- Hashing de senha com BCrypt (spring-boot-starter-security) — nunca armazenada em texto puro;
- Toggle em runtime (endpoint /admin/defense) para ligar/desligar a defesa sem reiniciar o container, permitindo o modo "comparar" do ataque-cli rodar os dois cenários em sequência;

- Allowlist de segurança (ataque-cli/seguranca.py) que restringe o alvo do ataque exclusivamente ao defesa-api dentro da rede do compose, tanto no CLI quanto no servidor HTTP do ataque-web;
- Dashboards (defesa-web e ataque-web) que exibem tentativas, IPs bloqueados e progresso do ataque em tempo quase real, tornando o efeito da defesa visível e comparável.

Com essa arquitetura, o mesmo ataque que esgota uma wordlist em segundos contra o alvo sem defesa é interrompido em poucas tentativas quando o rate limiting está ativo — demonstrando, de forma prática e reproduzível, o valor de um controle simples de tentativas frente a um ataque de força bruta.